

DOSSIER PROFESSIONNEL – Corentin Coulet

Titre professionnel visé : Concepteur Développeur d'Applications (RNCP 37873)

Modalité : Parcours de formation

Adresse : 5 avenue Branly, Saint Maurice de Beynost 01700

1. Présentation du contexte professionnel

Le présent dossier professionnel s'inscrit dans le cadre de la validation du titre professionnel *Concepteur Développeur d'Applications*. Il rassemble mes expériences, missions et réalisations effectuées au sein de la société **Ypok**, où j'occupe le poste de **développeur full-stack** depuis près de deux ans.

Ypok conçoit et maintient des outils logiciels destinés à la fonction publique et aux collectivités territoriales. L'environnement dans lequel j'évolue couvre des domaines très variés : **prise de rendez-vous pour CNI et passeports, gestion des demandes administratives, workflows d'instruction, communication en masse (courriel, SMS, courrier), gestion d'utilisateurs, gestion de régie, édition d'arrêtés comptables, billetterie, procédures personnalisées, planning et agendas.**

Mon rôle consiste à intervenir sur l'intégralité de la chaîne applicative :

- **UX/UI,**
- **front-end** (Symfony, jQuery, Vue, CSS),
- **back-end** (Symfony),
- **base de données** (MariaDB → transition PostgreSQL),
- **webservices,**
- **exports, génération PDF,**
- **mise à jour fonctionnelles,**
- **corrections de bugs complexes,**
- **documentation technique,**
- **refontes structurantes (ex : l'agenda complet).**

Je travaille en autonomie sur la majorité des tickets, tout en étant intégré à une équipe technique, fonctionnelle et projet. Ce dossier présente mes réalisations significatives au regard

des **trois activités-types** du titre CDA.

2. Rôle et responsabilités

Je suis développeur full-stack sur l'outil de GRU. J'interviens de bout en bout : UX/UI, développement front-end et back-end, modélisation de données, optimisation SQL, intégration de services tiers (SMS Factor), génération PDF, export JSON/YAML, migration de données, corrections et évolutions. Je travaille en autonomie quasi totale : les sujets me sont confiés sous forme de tickets, parfois précédés d'un bref brainstorming ; je prends ensuite en charge l'analyse, la conception technique, la réalisation, les tests, la mise en production et la documentation correspondante.

3. Environnement technique

- Backend : PHP / Symfony (socle historique incluant du 1.4 avec dette technique), architecture MVC, services applicatifs, Doctrine.
- Frontend : jQuery 1.12.4, jQuery UI 1.9.2 pour l'existant ; Symfony pour les écrans et composants (Vue.js pour certaines fonctionnalités spécifiques).
- Base de données : MariaDB (existant) avec une passation en cours vers PostgreSQL.
- Outillage : Git (branches par feature, revues), Docker, intégrations API (SMS Factor par exemple), génération PDF, exports JSON/YAML.
- Bibliothèques clés : FullCalendar (agenda), avec refonte lourde en cours pour dépasser ses limites sur des cas métiers complexes.

4. Exemples de pratique professionnelle (activités-types)

AT1 – Développer une interface utilisateur web statique (Refonte UX/UI GRU)

Contexte et périmètre

Le module GRU présentait une dette UX/UI marquée sur les écrans **Liste des demandes**, **Fiche dossier**, **Saisie/édition**, et **Récapitulatif PDF**. Les contraintes : conserver la logique fonctionnelle (règles de saisie, champs obligatoires, gabarits PDF existants) et **ne pas** modifier les APIs ni les workflows métier. Hors périmètre : moteur PDF, règles métiers, droits. Périmètre UI : **structure HTML**, **système de styles**, **composants** et **layouts**, en compatibilité avec jQuery 1.12.4 / jQuery UI 1.9.2.

Utilisateurs & parcours critiques

- **Agent guichet** (débutant) : ouvrir un dossier, vérifier l'identité, compléter un champ manquant, **changer le statut**.
- **Agent expérimenté** : filtrer une liste large, accéder aux **pièces jointes**, déclencher un **courrier**.
- **Référent** : contrôler rapidement les **dossiers en anomalie**.

Parcours cibles :

1. **Rechercher** → **Ouvrir dossier** → **Modifier champ** → **Enregistrer** → **Changer statut**
2. **Filtrer par statut** → **Trier par date** → **Exporter liste**
3. **Consulter dossier** → **Générer arrêté PDF** → **Télécharger/archiver**

Objectifs mesurables (critères d'acceptation)

- Temps pour trouver un statut de dossier ≤ 5 s sur écran 1366px.
- ≤ 3 **clics** pour passer du détail au changement de statut.
- Lisibilité valide à **125 % de zoom** et **contraste $\geq 4.5:1$** sur texte courant.
- Poids CSS de la page **réduit de 20 %** (fragmentation supprimée).
- **Ordre de tabulation** logique sur tous les champs essentiels.

Choix de conception UX/UI

- **Architecture CSS** : BEM + couches ITCSS légères (settings, tools, generic, elements, components, utilities).
- **Design tokens** (variables) : couleurs (primaire/secondaire/état), échelle typographique (12/14/16/20/24), **8-pt spacing** (4/8/16/24/32).
- **Grille & layout** : grille fluide 12 colonnes, **conteneur max 1280px**, zones latérales pour actions secondaires.
- **Composants normalisés** : Boutons (primaire/secondaire/danger), **Badge de statut** (nouveau/en cours/complet/anomalie), Alert/Toast, Onglets, Accordéons, Empty state, Pagination.

- **Hiérarchie visuelle** : titres H1/H2, regroupements, **alignement vertical des libellés/champs**, messages d'erreur **inline**.

Accessibilité (ciblage RGAA pragmatique)

- Contrastes validés (4.5:1 textes, 3:1 éléments UI).
- **Focus visibles** sur tous éléments interactifs, ordre tab logique, **Esc** ferme les modales.
- Libellés **for/id** systématiques, soutien **aria-live** pour confirmations.
- Textes de boutons **verbaux** (“Enregistrer les modifications”, “Changer le statut”).

Responsive & compatibilité

- Breakpoints : ≥ 1440 (dense), **1024–1439** (standard), ≤ 1024 (compact).
- Reflux de colonnes en pile, regroupement des actions en **kebab menu** en compact.
- Tests sur Chrome / Firefox / Edge (versions supportées des collectivités).

Tâches réalisées (détaillées)

- Audit des écrans existants et identification des points de friction UX.
- Conception d'une nouvelle structure HTML plus sémantique et plus modulaire.
- Refonte complète du CSS : normalisation des couleurs, composants, grilles, marges, typographies.
- Réorganisation des blocs fonctionnels : zones d'infos clés, onglets, listes, actions.
- Adaptation du responsive design : gestion des colonnes, collapses, breakpoints adaptés.
- Mise en place de premiers éléments d'accessibilité (contraste, navigation clavier, focus visibles).
- Revue interne avec les chefs de projet pour validation ergonomique.
- Intégration des maquettes statiques dans le socle existant.

Moyens techniques utilisés

- HTML5 / organisation sémantique
- CSS3 (refonte complète, responsive)
- Symfony 1.4 (on reste sur la même technologie mais en se focalisant sur l'UX)
- jQuery UI (compatibilité avec l'existant)
- PostCSS / outils internes
- Git, environnement Docker
- Documentation interne pour le suivi UX

Contraintes et difficultés rencontrées

- Nécessité de conserver une compatibilité avec un code historique (jQuery 1.12.4 + jQuery UI 1.9.2).
- CSS historique très fragmenté, difficile à factoriser.
- Absence de design system préalable.
- Contraintes légales : certains éléments (formulaires) devaient rester identiques fonctionnellement.
- Besoin de conserver des gabarits PDF cohérents avec la nouvelle interface.

Résultats obtenus

- Interface clarifiée, lisible et structurée.
- Baisse des erreurs de saisie rapportées par les agents.
- Meilleure compréhension des statuts de demande.
- Base CSS plus propre permettant de futures évolutions plus rapides.
- Retour utilisateur positif via les chefs de projet.

Compétences du référentiel démontrées

- Maquettage et structuration d'interfaces web statiques.

- Respect des normes HTML/CSS et notions d'accessibilité.
- Travail sur la cohérence graphique et ergonomique.
- Versionnement du code et intégration progressive dans un existant complexe.
- Analyse UX rapide et mise en œuvre d'améliorations concrètes.

AT2 – Développer une application web dynamique (Webservice SMS)

Contexte et périmètre

Les collectivités utilisent le GRU pour traiter les demandes d'usagers et gérer les rendez-vous. Elles avaient besoin d'un canal de communication fiable (SMS) pour :

- confirmer un dépôt de dossier,
- rappeler un rendez-vous (CNI, passeport...),
- diffuser une information urgente.

Le système historique ne permettait qu'un envoi manuel limité, sans suivi, sans logs, et sans contrôle des droits.

Le périmètre confié :

- conception + développement du webservice SMS complet,
- interface dynamique d'envoi,
- gestion des retours d'état + journalisation,
- intégration sécurisée dans l'existant,
- compatibilité multiservices.

Hors périmètre : facturation SMS, paramétrage avancé des templates.

Utilisateurs et parcours critiques

Profils :

- *Agent guichet* : envoi ponctuel d'un SMS de confirmation.
- *Agent instructeur* : envoi en masse lors de modifications d'horaires.
- *Administrateur* : suivi des quotas, analyse des erreurs.

Parcours cibles :

1. Dossier → “Notifier par SMS” → saisir message → valider → confirmation immédiate.
2. Sélection d’une liste de rendez-vous → envoi massif → suivi des statuts.
3. Panneau d’administration → filtrage par date/statut → audit complet.

Objectifs mesurables (critères d’acceptation)

- Temps entre clic “Envoyer” et retour d’état < **2 secondes**.
- Journalisation complète (acteur, message, statut, timestamp).
- Zéro message envoyé sans contrôle de permission.
- Affichage d’un message d’erreur clair sous **300 ms** lors d’un échec API.
- Taux de réussite d’envoi ≥ **98 %** (hors contraintes opérateurs).

Architecture et choix techniques

Backend :

- Service métier Symfony SmsService (envoi, vérification quotas, mapping statuts).
- Contrôleur interne /api/sms/send.
- Gestion des erreurs + retries exponentiels.
- Logs persistés en base (table sms_history).

Frontend :

- Formulaire dynamique jQuery :
 - Validation en temps réel (longueur, caractères interdits, champ obligatoire).
 - Affichage état “en cours / succès / échec”.

Intégration API SMS Factor :

- Appels REST authentifiés par token sécurisé en variable d’environnement.

- Normalisation interne des statuts opérateurs (DELIVERED / FAILED / UNKNOWN).

Tâches réalisées

- Analyse de la documentation API SMS Factor (webhooks, statuts, quotas).
- Conception et implémentation du service d'envoi (gestion des erreurs, retries).
- Création du contrôleur REST interne et des endpoints front.
- Développement de l'interface dynamique (jQuery + workflow utilisateur).
- Gestion des quotas : seuil par collectivité, alertes de dépassement.
- Mise en place d'un système complet de logs :
(agent, numéro, payload, statut, message retour API).
- Création d'un écran de suivi des envois (filtres : date, agent, statut).
- Mise en place d'un contrôle d'accès par rôle (ACL Symfony).
- Tests fonctionnels, tests API, simulations d'erreurs opérateurs.
- Documentation technique + guide d'usage.

Contraintes et difficultés

- **Limites de débit API** → file d'attente + throttling.
- **Erreurs réseau** → implémentation de stratégies retries + backoff.
- **RGPD** → minimisation des données stockées (pas de contenu long, pas d'infos sensibles).
- **Compatibilité multi-modules** → factorisation en un service transverse.
- **Interface simple malgré une logique complexe** (statuts, erreurs, quotas).

Sécurité & conformité

- Permissions basées sur les rôles agents (lecture / envoi / audit).
- Token API stocké en variable d'environnement (jamais en dur).
- Validation stricte du numéro (régex + normalisation E.164).

- Rétention limitée des logs selon la politique interne.
- Messages non persistés au-delà de la durée nécessaire.

Résilience et qualité

- Retraitement automatique en cas d'échec temporaire.
- Logs complets pour audit technique et fonctionnel.
- Alertes automatiques en cas de taux d'erreur élevé.
- Tests sur environnements isolés + sandbox SMS Factor.

Résultats obtenus

- Mise en production d'un outil utilisé quotidiennement par l'ensemble des collectivités.
- Visibilité complète sur les envois (audit + suivi des erreurs).
- Intégration fluide dans plusieurs modules sans régression.
- Très bon retour des agents (simplicité, rapidité).

Compétences du référentiel démontrées

- Développement dynamique (JS / jQuery / AJAX).
- Conception et développement back-end (Symfony, services métiers).
- Consommation d'API externes.
- Gestion des permissions, sécurité, conformité RGPD.
- Gestion d'erreurs, mécanismes de retries, résilience.
- Conception de workflows multi-étapes dans un existant complexe.
- Tests, documentation, intégration continue.

AT3 – Concevoir et développer une application multicouche (Refonte de l'agenda multicouche)

Contexte et périmètre

L'agenda historique du GRU reposait sur un FullCalendar configuré de manière minimale. Il

ne permettait pas :

- la gestion multi-ressources (agents, salles, matériel),
- les contraintes fortes (plages interdites, priorités),
- les récurrences complexes,
- la détection fiable de collisions,
- ni l'application de règles professionnelles propres aux collectivités.

Les évolutions demandées devenaient très difficiles à intégrer dans ce socle vieillissant.

La direction technique a donc lancé une **refonte complète du module agenda**, confiée en intégralité (backend, frontend, BDD, API, intégrations existantes).

Périmètre couvert :

- modélisation des données,
- services métier,
- contrôleurs/API,
- interface Vue.js,
- extension de FullCalendar,
- compatibilité progressive avec l'existant.

Hors périmètre :

moteur de notifications, synchronisation avec des agendas tiers (Google/Outlook), gestion fine des autorisations inter-services.

Utilisateurs & parcours critiques

Profils concernés :

- *Agent guichet* : réserver des créneaux pour les demandes CNI/passeport.
- *Administrateur d'agenda* : définir des règles de disponibilité, plages fermées, jours spéciaux.
- *Référent service* : suivre occupation, détecter les conflits, modifier une récurrence.

Parcours cibles :

1. Sélection d'une ressource → affichage du planning → ajout d'un créneau → vérification automatique → confirmation.
2. Création d'une récurrence hebdomadaire → détection des collisions → proposition d'ajustement.
3. Consultation rapide des indisponibilités → modification d'une plage → historique des modifications.

Objectifs mesurables (critères d'acceptation)

- Charger un planning complet < **1.5 s** sur un mois glissant.
- Retour de validation d'un créneau (accepté/refusé) < **300 ms**.
- Détection des collisions garantissant **0 conflit silencieux**.
- Une modification doit déclencher un audit systématique.
- Récurrences gérées jusqu'à **24 mois** sans dégradation notable.

Architecture cible

1. Base de données

Création d'un schéma relationnel dédié :

- ressource : agents, salles, équipements.
- timeslot : plages disponibles/indisponibles.
- event : créneaux réservés.
- rule : contraintes (fermatures, horaires obligatoires, jours spéciaux).
- audit_log : traçabilité.

Optimisation PostgreSQL :

- index B-Tree sur les dates,
- index GIN sur les données JSON (métadonnées d'événement),

- vues matérialisées pour les calculs récurrents.

2. Services métier

- Validation d'un créneau → règles simples + règles transverses.
- Détection de collisions → intervalle basé sur tsrange.
- Génération des récurrences → séries temporelles.
- Vérification de disponibilité multi-ressources.
- Application de règles conditionnelles selon la collectivité.

3. API REST

Endpoints structurés :

- GET /agenda/{resource}/events
- POST /agenda/event
- PUT /agenda/event/{id}
- DELETE /agenda/event/{id}
- POST /agenda/recurrence

Documentation internalisée via OpenAPI + schémas JSON.

4. Front-end (Vue.js + FullCalendar)

- Composants Vue.js :
Planning, Événement, Modale de création, Modale d'édition, Timeline.
- Customisation full FullCalendar :
hooks, rendering custom, eventSources multiples, tooltips dynamiques.

Tâches réalisées (détaillées)

- Analyse des besoins métiers en ateliers internes (collectivités, support, CP).
- Conception du nouveau modèle SQL (entités, relations, contraintes).
- Développement des services métier :
disponibilité, récurrences, collisions, règles avancées.

- Construction de l'API REST structurée et versionnée.
- Création d'une interface Vue.js dédiée (composants modulaires).
- Intégration avancée de FullCalendar (plugins spécifiques).
- Mise en place du système d'audit (toutes opérations CRUD sensibles).
- Tests sur les cas critiques :
 - transitions heure été/hiver,
 - chevauchements multi-ressources,
 - récurrences mensuelles variables.
- Documentation technique complète (diagrammes, API, décisionnels).

Contraintes et difficultés rencontrées

- **Dette technique** Symfony 1.4 / jQuery : nécessité d'isoler la nouvelle architecture dans un module plus moderne.
- **Limitations natives de FullCalendar** → contournées via hooks & rendering manuel.
- **Règles métiers variables selon collectivités**, parfois contradictoires → système de règles dynamiques.
- **Gestion complexe des fuseaux horaires**, particulièrement lors des changements UTC.
- **Charges SQL élevées** → optimisation lourde (index, requêtes segmentées).
- **Migration progressive** sans interrompre l'activité des services.

Résilience, performance et qualité

- Préchargement asynchrone des événements pour réduire le TTFB.
- Mise en cache ponctuelle (événements non modifiés).
- Découpage des requêtes longues en segments temporels.
- Stratégies anti-collisions garanties côté serveur (jamais côté front).
- Audit complet pour chaque modification.

Résultats obtenus

- **Planning plus fluide et modernisé**, adoption rapide par les agents.
- **Réduction drastique des collisions** grâce au moteur centralisé.
- Base technique permettant : ressources évolutives, nouvelles règles, synchronisation future.
- Complète maintenabilité (services isolés, code factorisé).
- Diminution des tickets liés aux incohérences d’agenda.

Compétences démontrées

- Conception d’architecture multicouche (données → services → API → front).
- Modélisation relationnelle avancée.
- Développement Vue.js avec interactions complexes.
- Optimisation SQL (indexation, PL/pgSQL, vues matérialisées).
- Intégration avancée d’une librairie tierce (FullCalendar).
- Maintenance évolutive dans un contexte contraint.
- Tests, documentation, démarche qualité.

5. Études de cas détaillées

5.1 Webservice d’envoi de SMS (SMS Factor)

Objectifs : offrir un canal de communication robuste pour confirmations de rendez-vous, rappels et informations importantes.

Conception : schémas de séquence, gestion des erreurs et des retries, quotas, limitation de débit, journalisation et suivi des statuts. Sécurité et conformité : contrôle d’accès par rôle, rétention configurable, minimisation des données lorsque cela est pertinent. Résultats : fiabilité accrue des communications et visibilité sur les livraisons, avec retour d’expérience positif des agents.

5.2 Exports JSON/YAML et interopérabilité

Objectifs : faciliter les échanges inter-systèmes (outils tiers des collectivités, sauvegardes, exploitation). Actions : schémas versionnés, endpoints d'export sécurisés, validations, exemples d'échantillons. Résultats : intégrations plus rapides et réduction des erreurs d'import côté partenaires.

5.3 Arrêtés comptables en PDF

Objectifs : produire des documents réglementaires homogènes et fiables. Actions : gabarits réutilisables, contrôle de complétude des champs obligatoires, mécanismes d'horodatage. Résultats : baisse des retours pour non-conformité et gain de temps pour les agents.

5.4 CRUD et workflows complexes

Objectifs : couvrir un large spectre d'objets métiers (demandes, usagers, régie, dossiers, RH). Actions : validations contextuelles, transitions d'état pilotées par services, sécurisation des chemins critiques, optimisations SQL (indexation, réécriture, pagination). Résultats : réduction des temps de réponse et plus grande stabilité fonctionnelle.

5.5 Refonte de l'agenda (multi-ressources)

Objectifs : offrir plus de souplesse que la librairie d'origine pour des contraintes complexes. Actions : nouveau modèle, services, API et composants ; stratégie de compatibilité progressive avec l'existant ; tests ciblés sur les cas limites (réurrences et chevauchements). Résultats : meilleure maintenabilité et adaptation aux besoins des collectivités.

6. Organisation, méthodes et collaboration

Mode opératoire : réception de tickets, qualification rapide, estimation, découpage en lots, réalisation incrémentale, démonstrations intermédiaires si nécessaire et déploiement maîtrisé. Contribution aux revues de code, documentation interne (notes techniques, guides de migration) et échanges techniques transverses.

7. Qualité, sécurité, performance et accessibilité

Qualité : revues de code, factorisation, conventions partagées, tests ciblés sur zones à risque. Sécurité : contrôle d'accès côté back-end, validations d'entrée, protection contre injections et XSS, gestion des secrets et cloisonnement. RGPD : minimisation, droits d'accès, rétention

adaptée, audit des opérations sensibles. Accessibilité : libellés explicites, contraste suffisant, focus visibles, alternatives textuelles sur les écrans refondus.

Performance : profilage de requêtes, indexation fine, pagination et chargement paresseux, caches applicatifs lorsque pertinent.

8. Difficultés majeures et résolutions

Bugs JS et comportements de librairies : reproduction isolée, patchs mesurés, documentation des contournements.

Requêtes SQL de grande taille : découpage fonctionnel, vues et matérialisation (PostgreSQL), indexation, analyse des plans d'exécution et réécritures ciblées.

Dette technique (Symfony 1.4) : encapsulation de l'héritage, adaptateurs, refactorings incrémentaux et tests de non-régression.

FullCalendar peu flexible : abstraction et composants Vue.js spécifiques, gestion d'états locaux et synchronisation contrôlée.

Conception interne limitée : formalisation légère (diagrammes, ADR), partages réguliers et capitalisation documentaire.

9. Indicateurs d'impact (illustratifs)

Temps de chargement des listes lourdes : réduction significative après optimisation SQL.

Taux de délivrance SMS : suivi avec retours d'état et classification des erreurs d'envoi.

Documents comptables : baisse des retours pour non-conformité.

Qualité logicielle : réduction des tickets de régression post-refontes.

10. Améliorations et perspectives

Finaliser la migration PostgreSQL et exploiter ses fonctionnalités avancées.

Renforcer les tests (unitaires, API, UI ciblés) et la couverture sur les parcours critiques.

Mettre en place un design system minimal et des composants UI réutilisables.

Poursuivre les améliorations d'accessibilité (ciblage RGAA pragmatique).

Entretenir une documentation vivante (ADR, guides de contribution).

11. Déclaration sur l'honneur

Je soussigné, Corentin Coulet, déclare sur l'honneur que les informations fournies dans ce dossier sont exactes et que je suis l'auteur des réalisations décrites.

Fait à Miribel,

le 14/11/2025.

Signature : *corentin coulet*